INSTITUT SUPÉRIEUR D'INFORMATIQUE
ET DE GESTION DE KAIROUAN

المعهد العالي للإعلامية
و التصرف بالقيروان

Université de Kairouan
جامعة القيروان

Chapitre 5: Les bases de données en Python

Enseignant responsable: Ghrab Sahar

Année universitaire: 2025/2026

BD relationnel

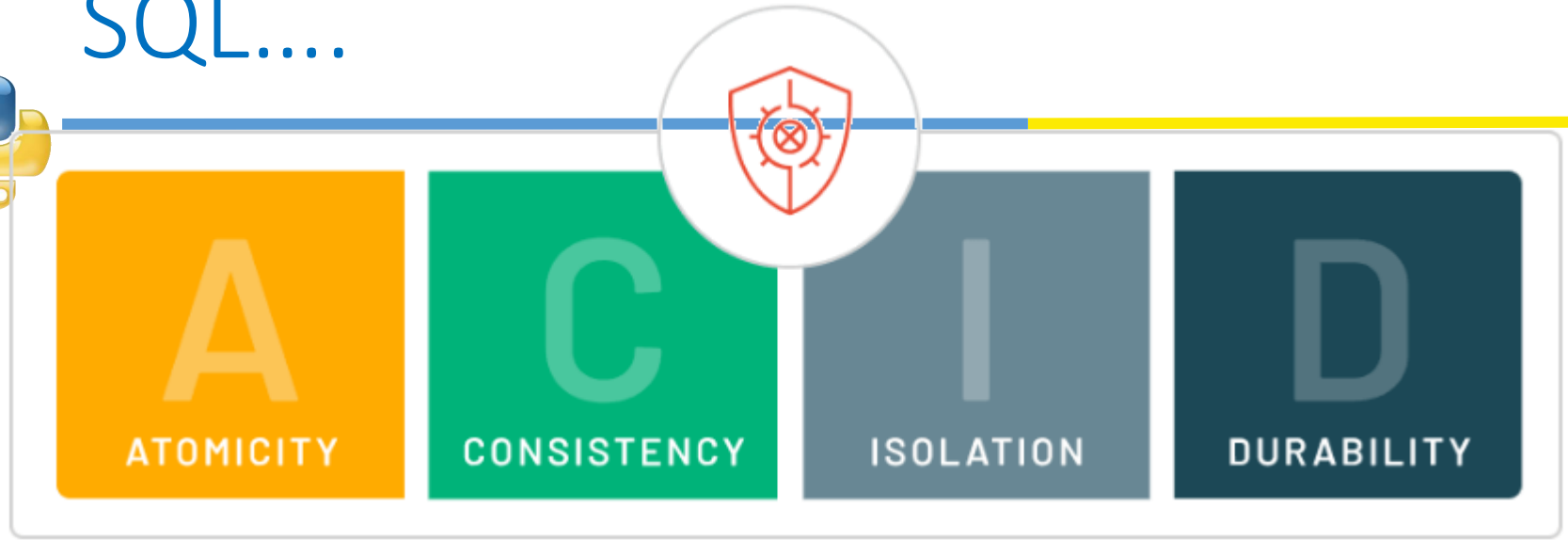
Base de données

- Une base de données (*database*) est un conteneur dans lequel il est possible de *stocker des données de façon structurée.* Cette structure permet au programme informatique connectée à celle-ci de faire des recherches complexes .
  - SQL est le langage d'interrogation des bases de données relationnelles.
  - Le SGBD est le logiciel de gestion de bases de données.

SQL….

- Un langage de requête plus ou moins normé.
- Toute information est décrite par des listes de n-uplets
- Opérations possibles: sélection (where), projection (select), produit cartésien (join), union
- Basé sur les propriétés **ACID (Atomicity, Coherence, Isolation, Durability)** qui sont un ensemble de propriétés qui garantissent qu'une transaction informatique est exécutée de façon fiable.
- Une opération sur les données est appelée une transaction ou transaction informatique

SQL....



Les modifications apportées aux données par les transactions réussies sont enregistrées, même en cas de défaillance du système.

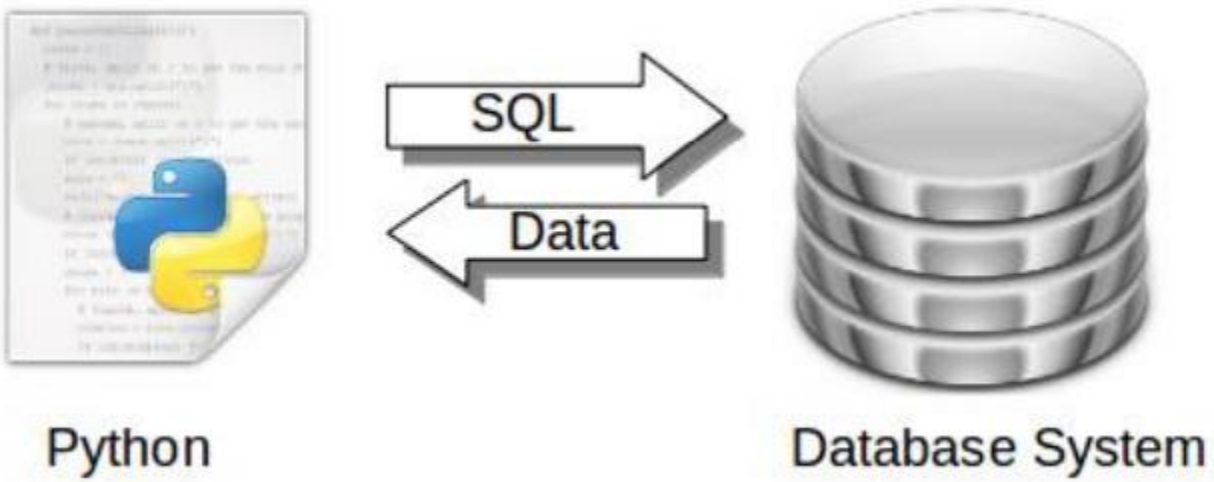
- Chaque instruction d'une transaction (lecture, écriture, mise à jour ou suppression de données) est traitée comme une seule unité.
- L'instruction est exécutée dans son intégralité ou elle ne l'est pas du tout.
- Cette propriété empêche que des données soient perdues ou corrompues.

- Les transactions ne modifient les tables que de façon prédéfinie et prévisible.
- La cohérence transactionnelle veille à ce qu'une corruption ou une erreur dans les données n'ait pas de conséquences inattendues sur l'intégrité de votre table.

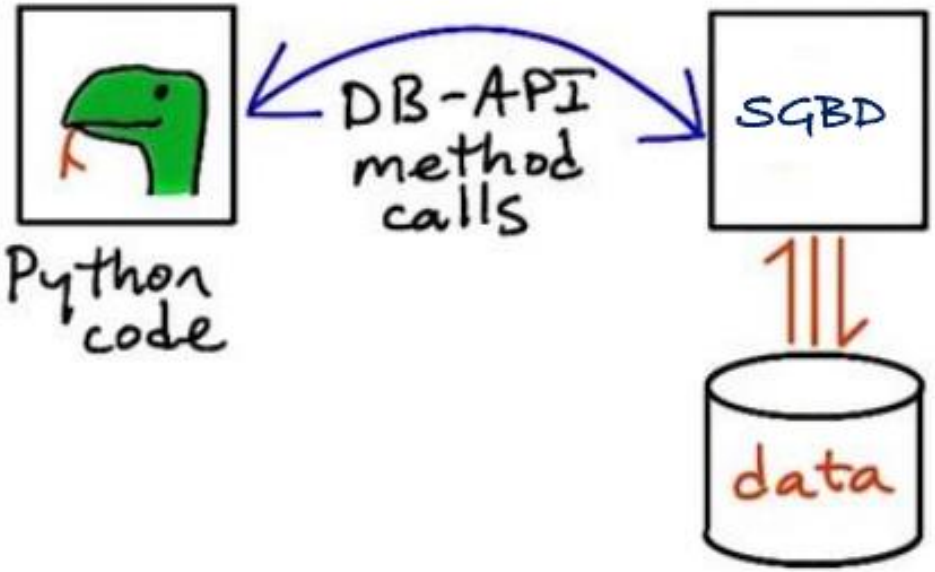
- Lorsque plusieurs utilisateurs lisent et écrivent dans la même table simultanément, l'isolement de leurs transactions garantit que les transactions effectuées simultanément ne s'impactent pas les unes les autres.
- Les requêtes sont exécutées comme si elles étaient successives alors qu'elles sont en réalité simultanées.

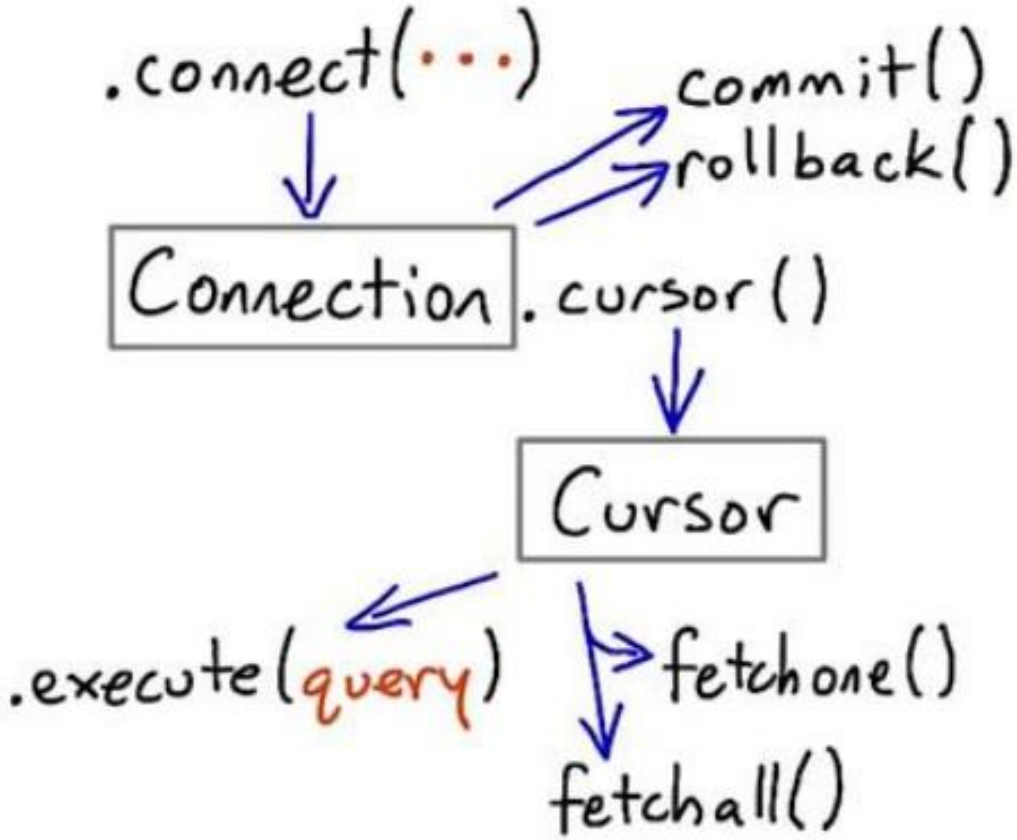
Accés à une BD en Python

- Il faut utiliser une interface standard pour les modules d'accès à une base de données ou Database Application Programming Interface (DB-API) de Python.



Accés à une BD en Python





Méthodes DB-API

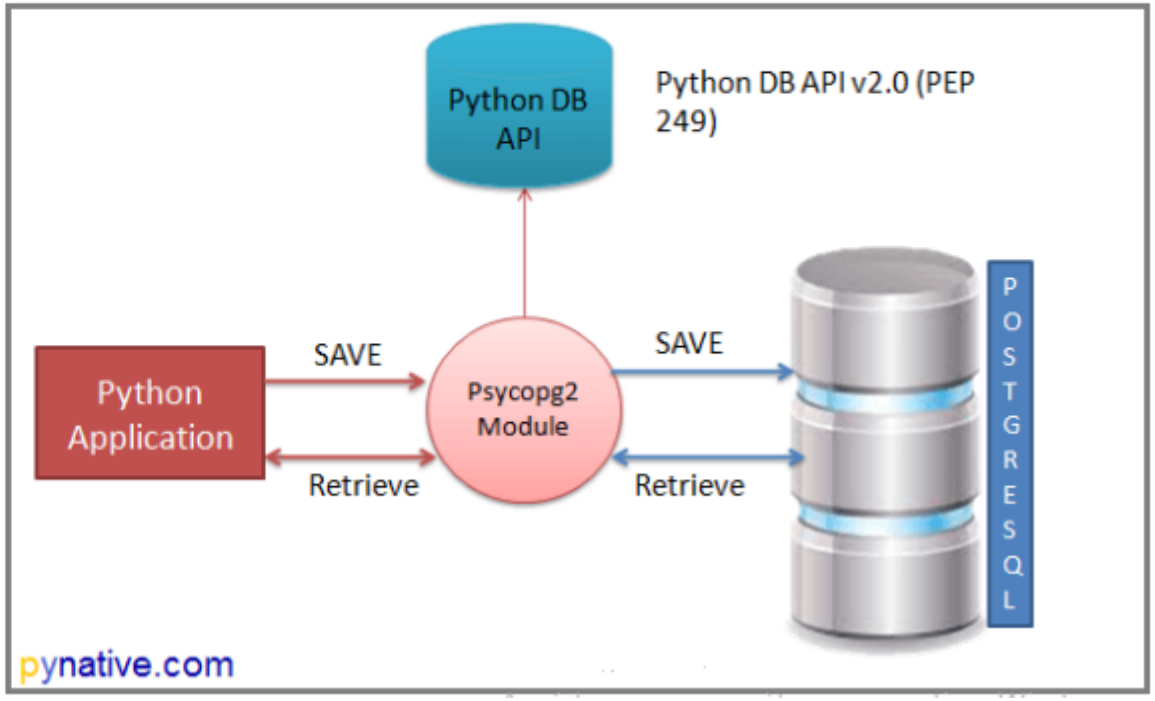
Méthode	Désignation
module.connect(...)	Se connecter au SGBD et à la base de données – la méthode renvoie un descripteur de connexion ou génère une exception
connection.cursor()	Créer un curseur pour exécuter une requête SQL et le cas échéant afficher le résultat
connection.commit()	Valider l'exécution d'une transaction (INSERT, UPDATE, DELETE)
connection.rollback()	Annuler une transaction (en cas d'exception par exemple)
connection.close()	Fermer la connexion
cursor.execute(statement)	Exécuter une requête
cursor.fetchall(), cursor.fetchone()	Récupérer les nuplets résultats d'une requête SELECT

DB-API Python

- `sqlite3` pour SQLite,
- `psycopg2` pour PostgreSQL
- `mysql-python` pour MySQL …

Pour installer le module :

```
>>> pip install psycopg2
```



Base de données

PostgreSQL	PostgreSQL est la base de données à utiliser pour les gros projets. Stable et très puissant, il permet de gérer des Gigabytes de données sans problème.
MySQL	Mysql est l'un des SGBDR les plus utilisés au monde. Il est gratuit et très puissant. Il possède la double licence GPL et propriétaire depuis son rachat par *Sun Microsystem* eux-mêmes racheté par *Oracle* (concurrent direct de MySQL). Le logiciel reste cependant entièrement gratuit et libre. Il répond à une logique client/serveur , c'est à dire que plusieurs clients (ordinateurs distants) peuvent se connecter sur un seul serveur qui héberge les données.
MariaDB	Le créateur de MySQL a crée MariaDB suite au rachat de MySQL pour continuer le projet en open source.
SQLite	SQLite est une bibliothèque écrite en C . SQLite est parfait pour les petits projets. Sa particularité est d'être intégré directement à un programme et ne répond donc pas à la logique client-serveur. Il est le moteur de base de données le plus distribué au monde puiqu'il est intégré à de nombreux logiciels grand public comme FireFox, Skype, Adobe, etc. Le logiciel pèse moins de 300 ko et peut donc être intégré à des projets tournant sur de petites supports comme les smartphones. Souvent aucune installation n'est nécessaire pour l'utiliser.
Oracle	Oracle Database est sous licence propriétaire, c'est à dire payant. Il est souvent utilisé pour les projets à gros budget nécessitant de réaliser des actions complexes.
Microsoft SQL Server	Produit *Microsoft* ne tourne que sur un OS Windows, payant n'apporte rien de plus que les logiciels concurrents libre de droit. Si vous avez trop d'argent à la limite...

DB-API Python (PostgreSQL)

```python
#cf https://librecours.net/module/dwh/etl01/pyt2c02.xhtml

import psycopg2

HOST = "localhost"
USER = "postgres"
PASSWORD = "****"
DATABASE = "postgres"

# Open connection
conn = psycopg2.connect("host=%s dbname=%s user=%s
password=%s" % (HOST, DATABASE, USER, PASSWORD))

# Open a cursor to send SQL commands
cur = conn.cursor()

# Testing
sql = "SELECT * FROM public.personne"
cur.execute(sql)
print(cur.fetchall())

#Close connection
conn.close()
```

`fetchall()` renvoie une liste de tuples :

```
>>> (executing file "<tmp 3>")
[(1, 'Gamotte', 'Albert'), (2, 'Pabien', 'Yvon'), (3,
'Computing', 'Claude'), (4, 'Slatable', 'Deborah'), (5
, 'Suffit', 'Sam'), (6, 'Debece', 'Gilles')]
```

DB-API Python (PostgreSQL)

Exemple d'utilisation de `fetchone()` qui renvoie un tuple :

```python
# Open a cursor to send SQL commands
cur = conn.cursor()

# Testing
sql = "SELECT * FROM public.personne"
cur.execute(sql)

# Fetch data line by line
raw = cur.fetchone()

while raw:
    print (raw[0])
    print (raw[1])
    print (raw[2])
    raw = cur.fetchone()

#Close connection
conn.close()
```

```
>>> (executing file "<tmp 3>")
1
Gamotte
Albert
2
Pabien
Yvon
3
Computing
Claude
4
Slatable
Deborah
5
Suffit
Sam
6
Debece
Gilles
```

Requêtes de mise à jour

Sans utiliser `commit()` :

```
# Open a cursor to send SQL commands
cur = conn.cursor()

sql = "INSERT INTO public.personne VALUES
(7,'Debece','Aude')"

cur.execute(sql)

# Testing
sql = "SELECT * FROM public.personne"
cur.execute(sql)
print(cur.fetchall())

#Close connection
conn.close()
```

Mise à jour OK en mémoire :

```
>>> (executing file "<tmp 3>")
[(1, 'Gamotte', 'Albert'), (2, 'Pabien', 'Yvon'), (3,
'Computing', 'Claude'), (4, 'Slatable', 'Deborah'), (5
, 'Suffit', 'Sam'), (6, 'Debece', 'Gilles'), (7, 'Debe
ce', 'Aude')]
```

Mais aucune modification sur le disque :

	pid [PK] integer	nom character varying (20)	prenom character varying (20)
1	1	Gamotte	Albert
2	2	Pabien	Yvon
3	3	Computing	Claude
4	4	Slatable	Deborah
5	5	Suffit	Sam
6	6	Debece	Gilles

Requêtes de mise à jour

En utilisant `commit()` :

```
# Open a cursor to send SQL commands
cur = conn.cursor()

sql = "INSERT INTO public.personne VALUES
(7,'Debece','Aude')"

cur.execute(sql)

# Testing
sql = "SELECT * FROM public.personne"
cur.execute(sql)
print(cur.fetchall())

# COMMIT!
conn.commit()

#Close connection
conn.close()
```

Mise à jour OK en mémoire :

```
>>> (executing file "<tmp 3>")
[(1, 'Gamotte', 'Albert'), (2, 'Pabien', 'Yvon'), (3,
'Computing', 'Claude'), (4, 'Slatable', 'Deborah'), (5
, 'Suffit', 'Sam'), (6, 'Debece', 'Gilles'), (7, 'Debe
ce', 'Aude')]
```

Répercussion de la mise à jour sur le disque :

	pid [PK] integer	nom character varying (20)	prenom character varying (20)
1	1	Gamotte	Albert
2	2	Pabien	Yvon
3	3	Computing	Claude
4	4	Slatable	Deborah
5	5	Suffit	Sam
6	6	Debece	Gilles
7	7	Debece	Aude

DB-API Python (SQLite)

Importation du module

```
import sqlite3
```

Création de la base de données SQLite

```
conn = sqlite3.connect('ma_base.db')
```

Création d'une base de données SQLite temporaire

```
conn = sqlite3.connect(':memory:')
```

Fermeture de la connexion vers la base

```
db.close()
```

DB-API Python (SQLite)

Création d'une table de la base

```
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS users(
     id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE,
     name TEXT,
     age INTERGER
)
""")
conn.commit()
```

Suppression d'une table de la base

```
cursor = conn.cursor()
cursor.execute("""
DROP TABLE users
""")
conn.commit()
```

DB-API Python (SQLite)

Insertion dans une table

```
cursor.execute("""
INSERT INTO users(name, age) VALUES(?, ?)""", ("olivier", 30))
```

Insertion dans une table à travers un dictionnaire

```
data = {"name" : "olivier", "age" : 30}
cursor.execute("""
INSERT INTO users(name, age) VALUES(:name, :age)""", data)
```

Récupération de l'id de la ligne que vous venez d'insérer

```
id = cursor.lastrowid
print('dernier id: %d' % id)
```

Plusieurs insertions en même temps avec la fonction executemany

```
users = []
users.append(("olivier", 30))
users.append(("jean-louis", 90))
cursor.executemany("""
INSERT INTO users(name, age) VALUES(?, ?)""", users)
```

DB-API Python (SQLite)

Récupération de la première ligne d'une requête à travers fetchone

```
cursor.execute("""SELECT name, age FROM users""")
user1 = cursor.fetchone()
print(user1)
```

Récupération des données de la requête à travers fetchall

```
cursor.execute("""SELECT id, name, age FROM users""")
rows = cursor.fetchall()
for row in rows:
    print('{0} : {1} - {2}'.format(row[0], row[1], row[2]))
```

L'objet curseur fonctionne comme un itérateur, invoquant la méthode fetchall() automatiquement

Recherche spécifique

```
id = 2
cursor.execute("""SELECT id, name FROM users WHERE id=?""", (id,))
response = cursor.fetchone()
```

Modification de données

```
cursor.execute("""UPDATE users SET age = ? WHERE id = 2""", (31,))
```

BD NoSQL

SQL non suffisant????????

- Évolution technologique: utilisation des systèmes distribués, données hétérogènes et hybrides, volume important.
- Les données Big Data ne correspondent pas aux propriétés ACID.
- Les systèmes Big Data favorisent la **distribution** et le **partitionnement**.
- Un systèmes est distribué utilise **‘n’ serveurs** qui vont interagir ensemble afin de stocker et de manipuler les données, c’est ce que l’on appelle un **“cluster”**.

Apparition d’un nouveau SGBD non relationnel
Apparition d’un nouvel langage spécifique ce type de SGBD

SQL non suffisant????????

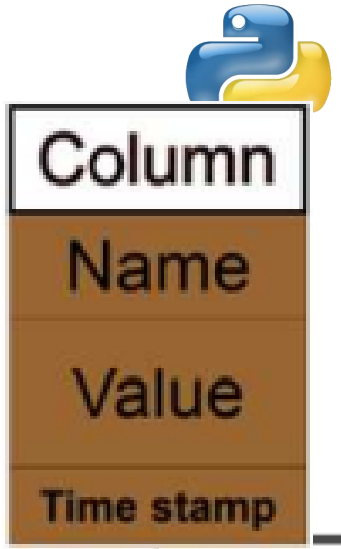
- Il existe de nombreuses technologies permettant de stocker et de manipuler des grosses volumétries de données :
  - les SGBDR distribués qui vont garantir les propriétés ACID énoncées précédemment,
  - les bases Not only SQL (NoSQL),
  - les bases New SQL
  - les systèmes de stockage fichier (exemple : Hadoop HDFS).

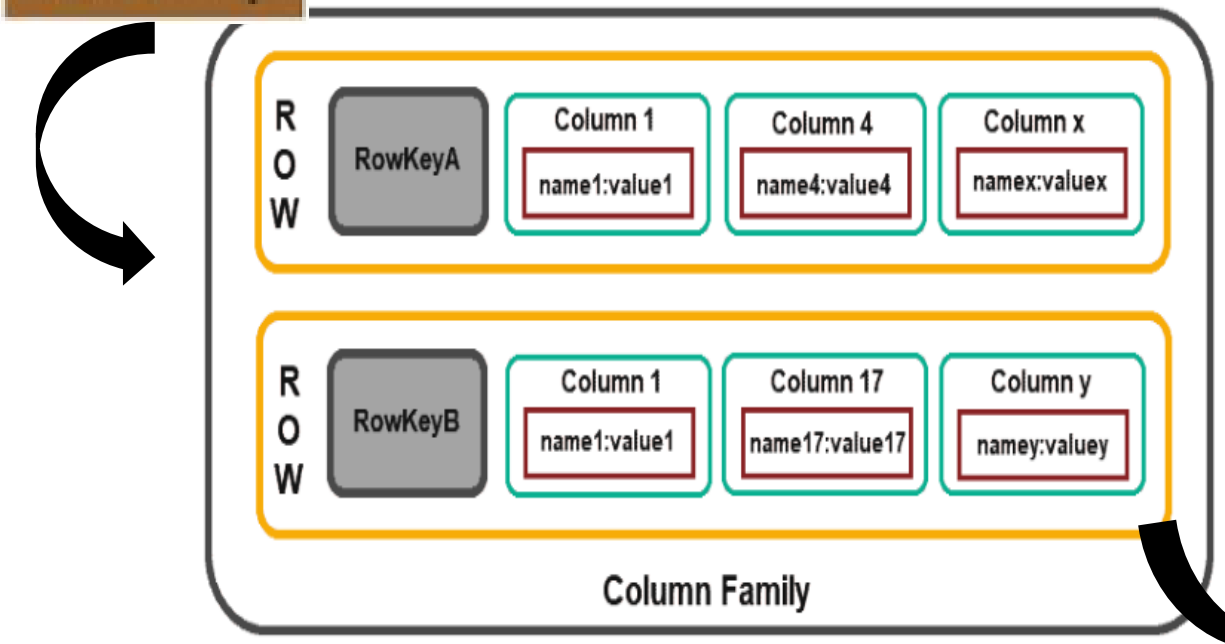
BD NoSQL

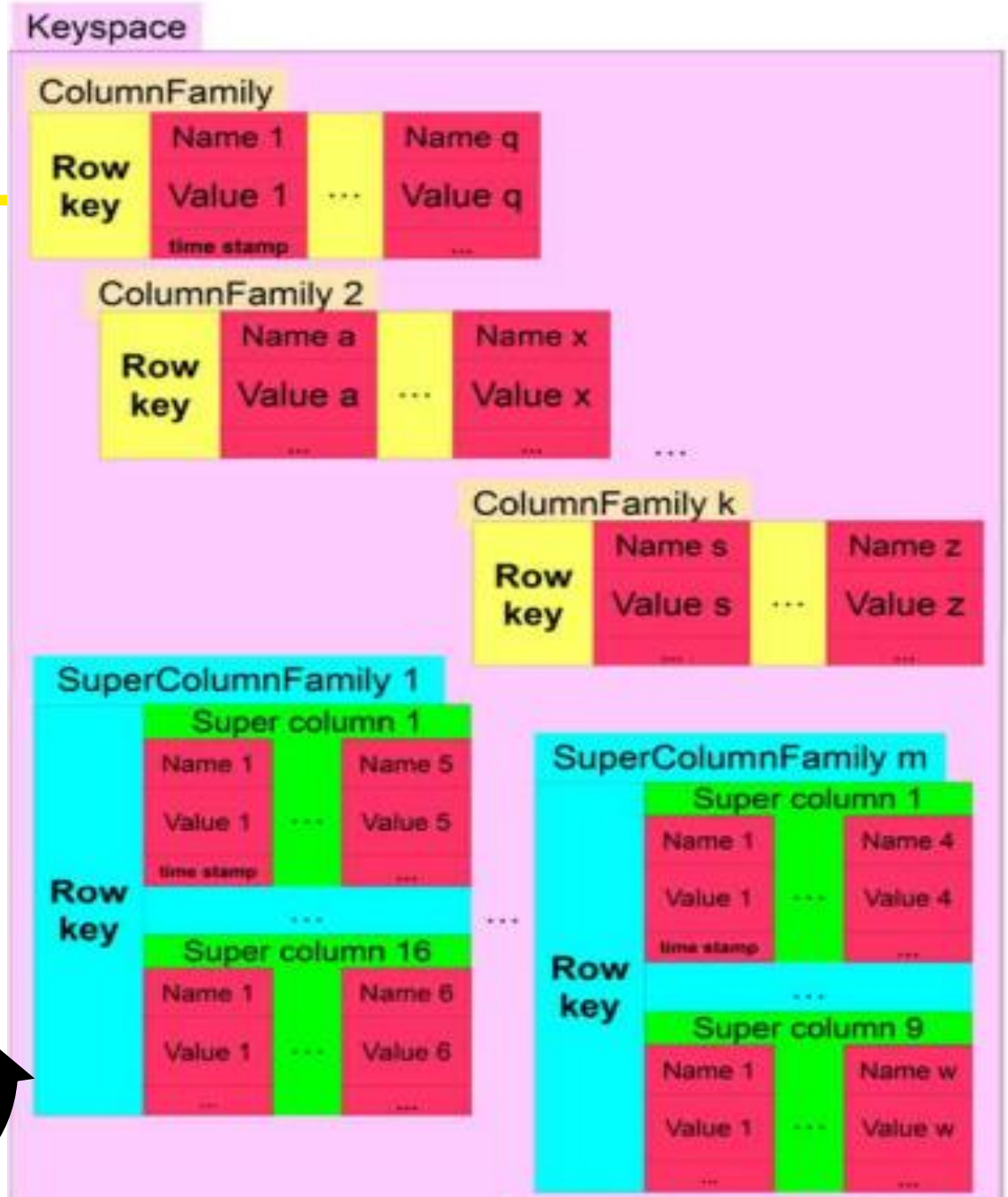
- Différents types de base de données NoSQL:
  - Colonne
  - Clé/valeur
  - Document
  - Graphe, etc
- Les types Document et Graphe sont basés sur le type Clé/Valeur

Type	Example
Key-Value Store	redis, riak
Wide Column Store	HBASE, cassandra
Document Store	mongoDB, CouchDB relax
Graph Store	Neo4j, InfiniteGraph The Distributed Graph Database

BD NoSQL Colonne







BD NoSQL Colonne

Exemple : Représentation de ventes en relationnel

Table Sales

#ticket	#date	#book
1	01/01/16	2212121504
1	01/01/16	2212141556
2	01/01/16	2212141556

Table Book

#isbn	#title	#author
2212121504	Scenari	1
2212141556	NoSQL	2

Table Author

#id	surname	firstname
1		
2	Bruchez	Rudi

Exemple : Représentation de ventes en colonne

Family Sales

#ticket	date	books
1	01/01/16	2212121504 2212141556
2	01/01/16	2212141556

Family Book

#isbn	title	a-surname	a-firstname
2212121504	Scenari		
2212141556	NoSQL	Bruchez	Rudi

BD NoSQL Document-Exemple

Exemple : Représentation de ventes en relationnel

Table Sales

#ticket	#date	#book
1	01/01/16	2212121504
1	01/01/16	2212141556
2	01/01/16	2212141556

Table Book

#isbn	#title	#author
2212121504	Scenari	1
2212141556	NoSQL	2

Table Author

#id	surname	firstname
1		
2	Bruchez	Rudi

Exemple : Représentation de ventes en document

Collection Sales

#oid	
4d0407660766b236450b45a3	"ticket" : 1 "date" : "01/01/16" "books" : ["_id" : 2212121504 "_id" : 2212141556]
4d0407660766b236450b45a4	"ticket" : 2 "date" : "01/01/16" "books" : ["_id" : 2212141556]

Collection Book

#oid	
4d0407660766b236450b45a5	"isbn" : 2212121504 "title" : "Scenari"

4d0407660766b236450b45a6	"isbn" : 2212141556
	"title" : "NoSQL"
	"author" : {
	"surname" : Bruchez
	"firstname" : Rudi
	}

BD NoSQL Document-Exemple

Exemple : Représentation de ventes en relationnel

Table Sales

#ticket	#date	#book
1	01/01/16	2212121504
1	01/01/16	2212141556
2	01/01/16	2212141556

Table Book

#isbn	#title	#author
2212121504	Scenari	1
2212141556	NoSQL	2

Table Author

#id	surname	firstname
1		
2	Bruchez	Rudi

Exemple : Représentation de ventes en document (avec imbrication redondante)

Collection Sales

#oid	

4d0407660766b236450b45a3	"ticket" : 1
	"date" : "01/01/16"
	"books" : [
	{
	"isbn" : 2212121504
	"title" : "Scenari"
	}
	{
	"isbn" : 2212141556
	"title" : "NoSQL"
	"author" : {
	"surname" : Bruchez
	"firstname" : Rudi
	}
	}
	]

4d0407660766b236450b45a4	"ticket" : 2
	"date" : "01/01/16"
	"books" : [
	{
	"isbn" : 2212141556
	"title" : "NoSQL"
	"author" : {
	"surname" : Bruchez
	"firstname" : Rudi
	}

BD NoSQL Graphe-Exemple

Exemple : Représentation de ventes en relationnel

Table Sales

#ticket	#date	#book
1	01/01/16	2212121504
1	01/01/16	2212141556
2	01/01/16	2212141556

Table Book

#isbn	#title	#author
2212121504	Scenari	1
2212141556	NoSQL	2

Table Author

#id	surname	firstname
1		
2	Bruchez	Rudi

Classe Sales

#oid		
4d0407660766b236450b45a3	*property*	ticket : 1
	property	date : 01/01/16
	relation	book : 4d0407660766b236450b45a5
	relation	book : 4d0407660766b236450b45a6
4d0407660766b236450b45a4	*property*	ticket : 2
	property	date : 01/01/16
	relation	book : 4d0407660766b236450b45a6

Classe Book

#oid		
4d0407660766b236450b45a5	*property*	title : Scenari
4d0407660766b236450b45a6	*property*	title : NoSQL
	relation	author : 4d0407660766b236450b45a8

Classe Author

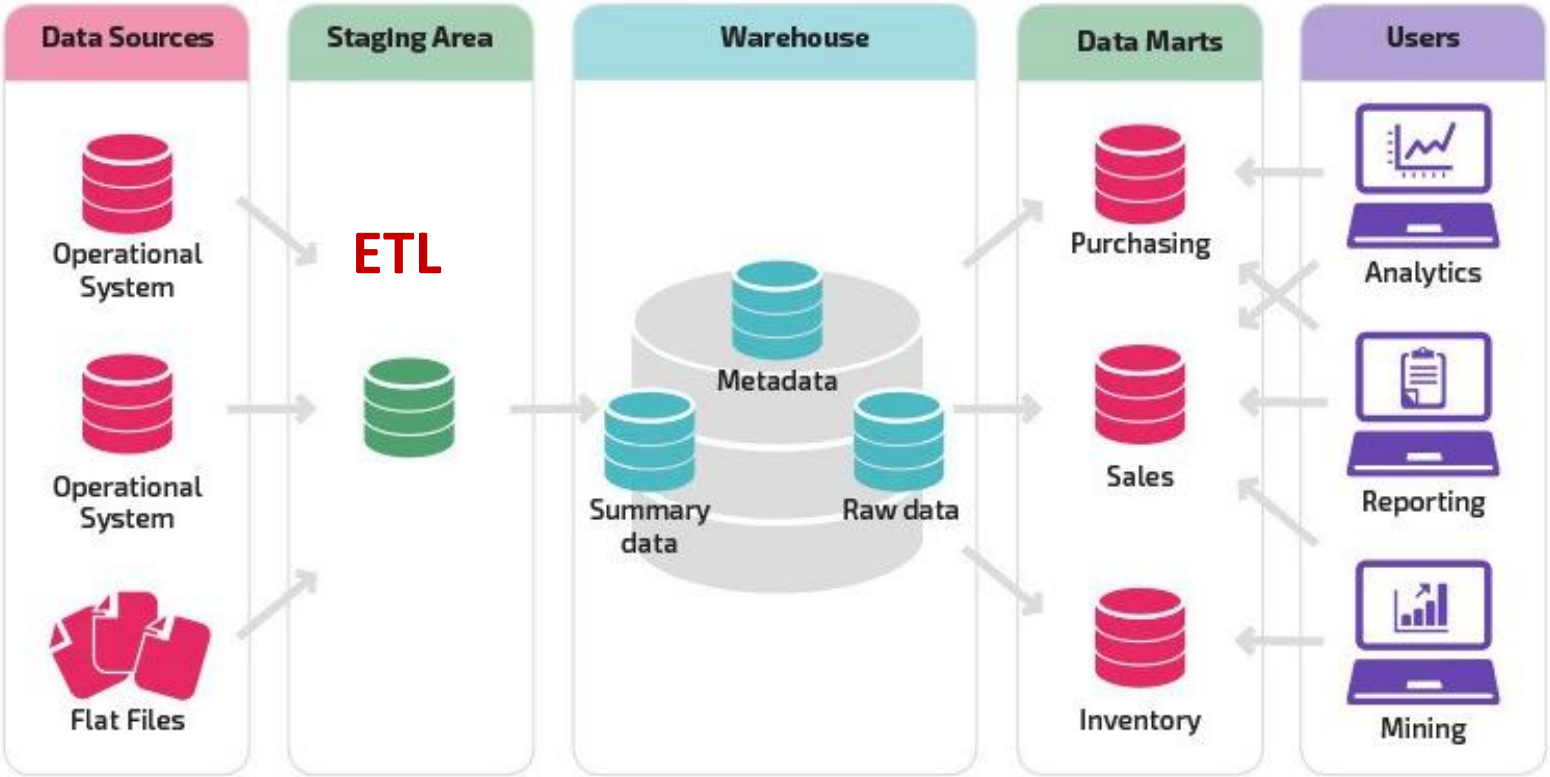
#oid		
4d0407660766b236450b45a8	*property*	surname : Bruchez
	property	firstnam : Rudi

Comparaison entre types de BD NoSQL

Modèle	Performance	Évolutivité	Flexibilité	Complexité	Fonctionnalité
Clé/Valeur	Élevée	Élevée	Élevée	Aucune	Variable (Aucune)
Colonne	Élevée	Élevée	Modérée	Faible	Minimale
Document	Élevée	Variable	Élevée	Faible	Variable (Faible)
Graphe	Variable	Variable	Élevée	Grande	Théorie des graphes

Entrepôt de données (ED)

- L'ED est une collection de données orientées sujet, intégrées, non volatiles, historisées et organisées pour la prise de décision,
- C'est une version centralisée et universelle de toutes les informations de l'entreprise.
- C'est une structure (comme une BD) qui a pour but, contrairement aux bases de données de l'entreprise pour des fins analytiques et pour aider à la décision stratégique.



Processus ETL

- **Extraction(E):** Lecture et récupération des données depuis les systèmes sources (bases de données relationnelles, fichiers CSV, API, etc.).
- **Transformation (T):** Application de règles de nettoyage, d'harmonisation, de calcul et d'agrégation des données pour les préparer à l'analyse (ex : conversion d'unités, jointures, gestion des valeurs manquantes).
- **Chargement (L) :** Écriture des données transformées dans le système cible (souvent le Data Warehouse), en respectant le schéma de données (tables de faits et de dimensions).

PygramETL

- pygramETL est un package Python conçu spécifiquement pour la mise en œuvre de processus ETL de manière structurée et efficace. pour créer des programmes Extract-Transform-Load (ETL) en Python.
- L'installation de ce package est disponible via ce lien: http://https://pygrametl.org/
- Le paquet contient:
  - Un certain nombre de classes pour remplir les tabes de faits et les dimensions.
  - Des classes pour extraire des données de différentes sources,
  - Des classes pour définir des étapes dans un flux ETL
  - Des fonctions pratiques pour la fonctionnalité ETL souvent nécessaires.

PygramETL

- Un package est un dossier contenant des sous-packages ou des modules.
- Un module est un fichier Python .py individuel.
- Les modules du package sont :
  - **datasources** : pour accéder à différentes sources de données
  - **tables** : pour fournir un accès simple et abstrait aux tables de dimensions et aux tables de faits
  - **parallel** : pour paralléliser un flux ETL
  - **JDBCConnectionWrapper** et **jythonmultiprocessing** : pour le support de Jython
  - **aggregators** : pour agréger des données
  - **steps** : pour définir les étapes d'un flux ETL
  - **FIFODict** : un dictionnaire de taille limitée dans lequel les éléments sont supprimés selon une logique *premier entré, premier sorti* (FIFO)
  - **drawntabletesting** : pour tester un flux ETL

https://pygrametl.org/doc/quickstart/beginner.html

PygramETL- Extraction

- pygramETL prend en charge de nombreuses sources de données qui sont déplacées dans un dictionnaire appelé row.
- Chacune des sources de données de cette classe est itérable.
- La récupération des données dans PygramETL se fait à travers une source de données qui doit être itérable:

```
for row in databasource
```

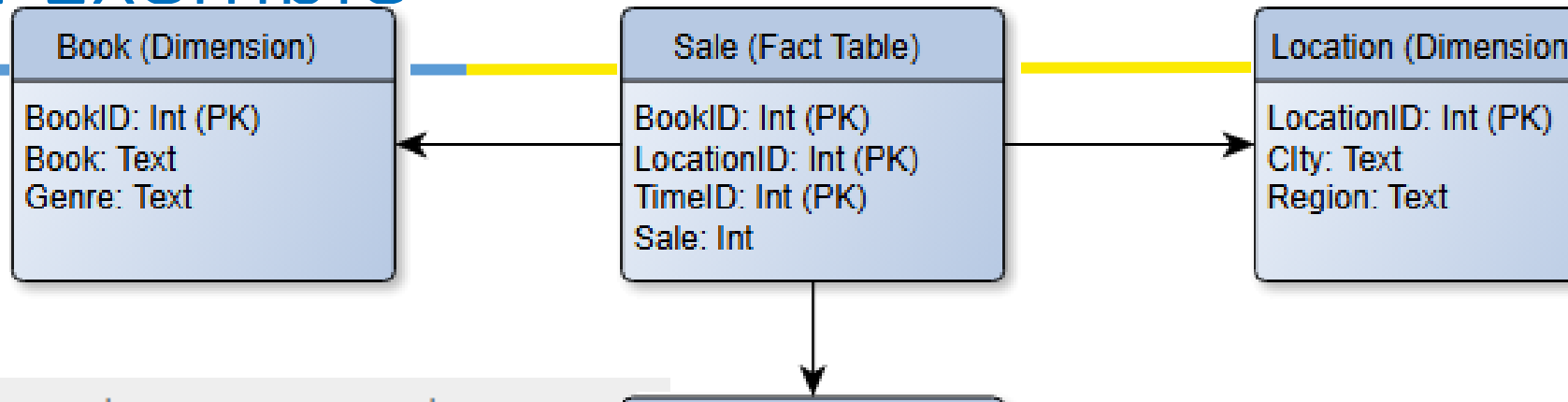
PygramETL- Types de table de dimension

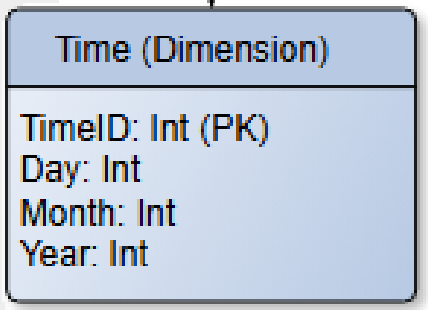
- CachedDimension: un cache pour réduire la latence des recherches en réduisant le nombre d'allers-retours vers la base de données.
- BulkDimension: une dimension optimisée pour un débit élevé lors de l'insertion de lignes et des recherches rapides.
- CachedBulkDimension: très similaire à BulkDimension et est également destinée au changement en bloc d'une dimension, de sorte que seules leurs différences sont décrites ici.
- TypeOneSlowlyChangingDimension: permet la création d'une dimension de type 1 à évolution lente.
- SlowlyChangingDimension: permet la création soit d'une dimension à évolution lente de type 2 soit dune dimension combinée à évolution lente de type 1 et de type 2.
- SnowflakedDimension: représente une dimension floconnée (normalisée) en tant qu'objet unique avec la même interface que Dimension.

PygramETL- table de faits

- La classe responsable à effectuer des opérations sur la table des faits est la classe FactTable
- Il requiert qu'une table appropriée ait été crééé dans la BD
- Une connexion la BD doit être crée l'aide d'une instance de la classe ConnectionWrapper
- Une table de fait peut être
  - **BatchFactTable**: charge les faits dans la table des faits par lots au lieu d'un à la fois comme FactTable
  - **BulkFactTable**: insère des faits par lots mais écrit les faits dans un fichier temporaire au lieu de les conserver en mémoire.

PygramETL-Exemple






```
book:text                | genre:text | store:text | date:date      | sale:int
-------------------------| ---------- | ---------- | -------------- | --------
Nineteen Eighty-Four     | Novel      | Aalborg    | 2005/08/05     | 50
Calvin and Hobbes One    | Comic      | Aalborg    | 2005/08/05     | 25
The Silver Spoon         | Cookbook   | Aalborg    | 2005/08/14     | 5
The Silver Spoon         | Cookbook   | Odense     | 2005/09/01     | 7
....                     |            |            |                |
```

```
city,        region
Aalborg,     North Denmark Region
Odense,      Region of Southern Denmark
```

PygramETL-Exemple

Importation de la bibliothèque

Importation de pygramETL

```python
# psycopg2 is a database driver allowing CPython to access PostgreSQL
import psycopg2

# sqlite3 is a database driver allowing CPython to access SQLite
import sqlite3

# pygrametl's __init__ file provides a set of helper functions and more
# importantly the class ConnectionWrapper for wrapping PEP 249 connections
import pygrametl

# pygrametl makes it simple to read external data through datasources
from pygrametl.datasources import SQLSource, CSVSource

# Interacting with the dimensions and the fact table is done through a set
# of classes. A suitable object must be created for each table
from pygrametl.tables import CachedDimension, FactTable
```

PygramETL-Exemple

Connexion à une BD SQLite

Connexion à une BD PotgreSQL

Construction du wrapper

```
# Creates a PEP 249 connection to the sales database. PARSE_DECLTYPES makes
# sqlite3 return values with the types specified in the database's schema
sale_conn = sqlite3.connect("sale.sqlite",
        detect_types=sqlite3.PARSE_DECLTYPES)

# While SQLite is used in this guide, any RDBMS that provides a PEP 249
# driver can be used with pygrametl. For example, SQLite can be replaced
# with PostgreSQL by simply replacing sale_conn with following two lines
# sale_string = "host='localhost' dbname='sale' user='user' password='pass'"
# sale_conn = psycopg2.connect(sale_string)

# A connection is also created to the data warehouse. The connection is
# then given to a ConnectionWrapper so it becomes implicitly shared between
# all the pygrametl abstractions that needs it without being passed around
dw_string = "host='localhost' dbname='dw' user='dwuser' password='dwpass'"
dw_conn = psycopg2.connect(dw_string)

# Although the ConnectionWrapper is shared automatically between pygrametl
# abstractions, it is saved in a variable so the connection can be closed
dw_conn_wrapper = pygrametl.ConnectionWrapper(connection=dw_conn)
```

PygramETL-Exemple

Préparation de la requête

Chargement des données à partir du fichier csv

```
# The location dimension stores the name of a location in the column city
# instead of in the column store as done in the input data from the sales
# database. By passing SQLSource a sequence of names matching the number of
# columns in the table it can automatically rename the columns
name_mapping = 'book', 'genre', 'city', 'date', 'sale'

# Extraction of rows from a database using a PEP 249 connection and SQL
query = "SELECT book, genre, store, date, sale FROM sale"
sale_source = SQLSource(connection=sale_conn, query=query,
        names=name_mapping)

# Extraction of rows from a CSV file does not require a PEP 249 connection,
# just an open file handler. pygrametl uses Python's DictReader for CSV
# files and assumes the header of the CSV file contains the name of each
# column. When using CSVSource it is very important to convert the values
# to the correct type before inserting them into a table through pygrametl.
# As region.csv is encoded as UTF-8 and contains the non-ASCII characters æ
# and ø, the encoding open() will use is explicitly set to UTF-8. Of course,
# if a file uses a different encoding than UTF-8 it should be used instead
region_file_handle = open('region.csv', 'r', 16384, "utf-8")
region_source = CSVSource(f=region_file_handle, delimiter=',')
```

PygramETL

Construction de la dimension book

Construction de la dimension time

Construction de la dimension location

```
# An instance of CachedDimension is created for each dimension in the data
# warehouse. CachedDimension uses a local cache to significantly reduce the
# number of requests issued to the RDBMS. CachedDimension should generally
# be used instead of Dimension unless the higher memory consumption causes
# problems. For each dimension, the name of the database table, the table's
# primary key, and the table's non-key columns (attributes) are given. In
# addition, for the location dimension, the subset of the attributes that
# should be used to lookup the primary key is given. As mentioned in the
# beginning of this guide, using named parameters is strongly encouraged
book_dimension = CachedDimension(
        name='book',
        key='bookid',
        attributes=['book', 'genre'])

time_dimension = CachedDimension(
        name='time',
        key='timeid',
        attributes=['day', 'month', 'year'])

location_dimension = CachedDimension(
        name='location',
        key='locationid',
        attributes=['city', 'region'],
        lookupatts=['city'])
```

```
# A single instance of FactTable is created for the data warehouse's single
# fact table. It is created with the name of the table, a list of columns
# constituting the primary key of the fact table, and a list of measures
fact_table = FactTable(
        name='sale',
        keyrefs=['bookid', 'locationid', 'timeid'],
        measures=['sale'])
```

Construction de la table fait

PygramETL-Exemple

Fonction split_date

```
# A normal Python function is used to split the date into its parts
def split_date(row):
    """Splits a date represented by a datetime into its three parts"""

    # First the datetime object is extracted from the row dictionary
    date = row['date']

    # Then each part is reassigned to the row dictionary. It can then be
    # accessed by the caller as the row is a reference to the dict object
    row['year'] = date.year
    row['month'] = date.month
    row['day'] = date.day
```

PygramETL-E

Insertion de lignes dans une dimension

Appel de la fonction split_date

```
# The location dimension is filled with data from the CSV file as the file
# contains all the information required for both columns in the table. If
# the dimension was filled using data from the sales database, it would be
# necessary to update the region attribute with data from the CSV file
# later. To insert the rows the method CachedDimension.insert() is used
[location_dimension.insert(row) for row in region_source]

# The file handle to the CSV file can then be closed
region_file_handle.close()

# All the information needed for the other dimensions are stored in the
# sales database. So with only a single iteration over the sales records
# the ETL flow can split the date and lookup the three dimension keys
# needed for the fact table. While retrieving the dimension keys, pygrametl
# can automatically update the dimensions with new data if ensure() is
# used. This method combines a lookup with an insertion so a new row is
# only inserted into the dimension or fact table if it does not yet exist
for row in sale_source:

    # The date is split into its three parts
    split_date(row)

    # The row is updated with the correct primary keys for each dimension, and
    # any new data are inserted into each of the dimensions at the same time
    row['bookid'] = book_dimension.ensure(row)
    row['timeid'] = time_dimension.ensure(row)
```

PygramETL-

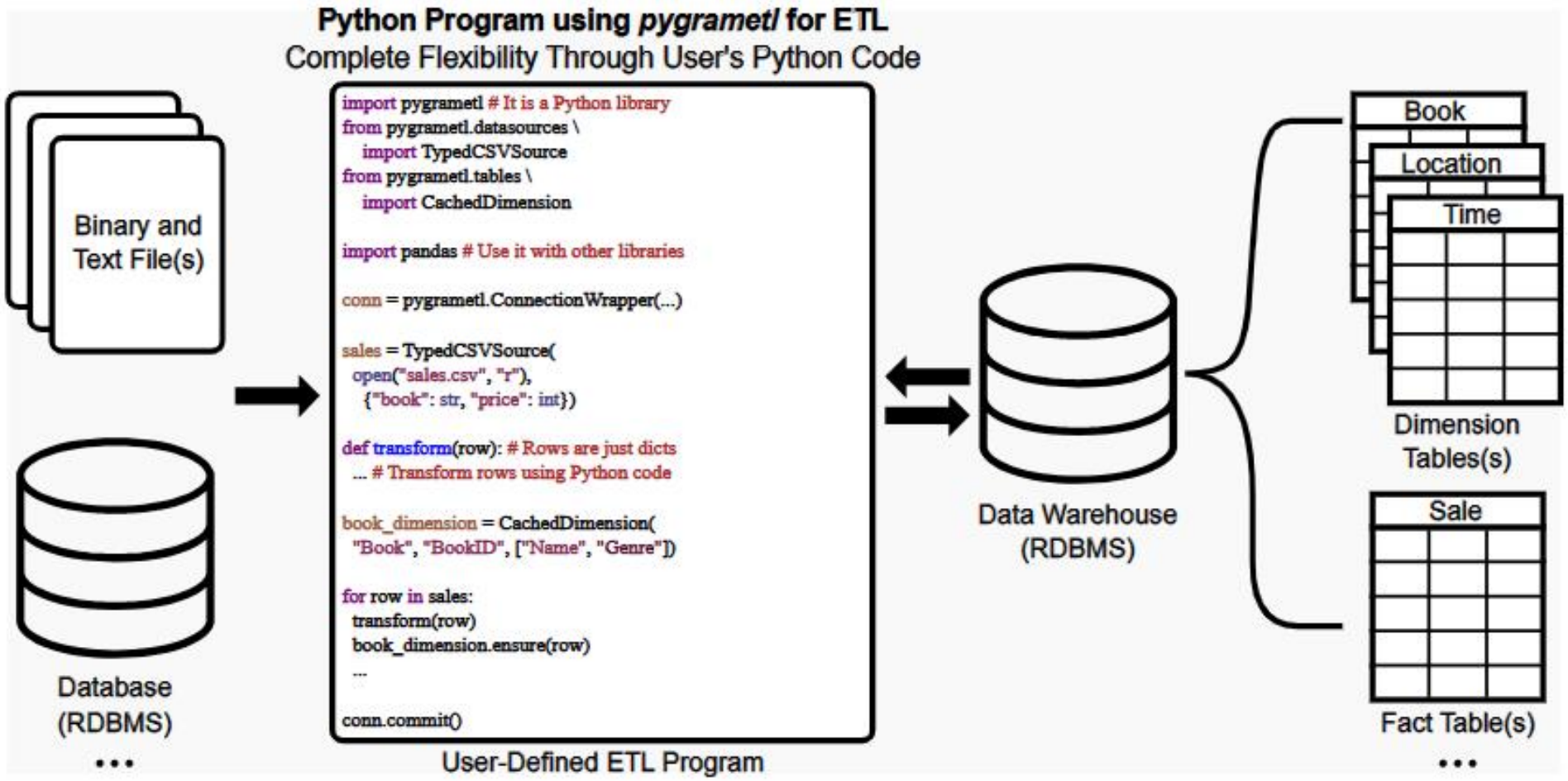
```
    # CachedDimension.ensure() is not used for the location dimension as it
    # has already been filled. Instead the method CachedDimension.lookup()
    # is used. CachedDimension.lookup() does not insert any data and
    # returns None if a row with the correct lookupatts is not available.
    # This makes error handling very simple to implement. In this case an
    # error is raised if a location is missing from the CSV file as
    # recovery is not possible
    row['locationid'] = location_dimension.lookup(row)
    if not row['locationid']:
        raise ValueError("city was not present in the location dimension")

    # As the number of sales is already aggregated in the sales records, th
    # row can now be inserted into the data warehouse. If aggregation, or
    # other more advanced transformations are required, the full power of
    # Python is available as shown with the call to split_date()
    fact_table.insert(row)

# After all the data have been inserted, the connection is ordered to
# commit and is then closed. This ensures that the data is committed to the
# database and that the resources used by the connection are released
dw_conn_wrapper.commit()
dw_conn_wrapper.close()

# Finally, the connection to the sales database is closed
sale_conn.close()
```

Insertion dans la table de faits

Validation et fermeture du wrapper

Fermeture de la base

PygramETL


Python Program using *pygrametl* for ETL
Complete Flexibility Through User's Python Code
Binary and Text File(s)
Database (RDBMS)
...
import pygrametl # It is a Python library
from pygrametl.datasources \
import TypedCSVSource
from pygrametl.tables \
import CachedDimension
import pandas # Use it with other libraries
conn = pygrametl.ConnectionWrapper(...)
sales = TypedCSVSource(
open("sales.csv", "r"),
{"book": str, "price": int})
def transform(row): # Rows are just dicts
... # Transform rows using Python code
book_dimension = CachedDimension(
"Book", "BookID", ["Name", "Genre"])
for row in sales:
transform(row)
book_dimension.ensure(row)
...
conn.commit()
User-Defined ETL Program
Data Warehouse (RDBMS)
Book
Location
Time
Dimension Tables(s)
Sale
Fact Table(s)
...

Exercice 1

Créer un programme qui demande à l'utilisateur un nom et une note, en boucle. Les résultats sont stockés au fur et à mesure dans une base de données. Si le nom est égal à «Q» ou «q», le programme s'arrête.

```
Shell
>>> %Run dfs.py
  Nom ? Alice
  Note ? 12
  Nom ? Bob
  Note ? 3
  Nom ? q
>>> |
```

```python
import sqlite3

#Connexion
connexion = sqlite3.connect('mabasecobaye.db')

#Récupération d'un curseur
c = connexion.cursor()

c.execute("""
    CREATE TABLE IF NOT EXISTS notes(
    Nom TEXT,
    Note INT);
    """)


while True :
    nom = input('Nom ? ')
    if nom in ['Q','q'] :
        break
    note = input('Note ? ')
    data = (nom, note)
    p = "INSERT INTO notes VALUES ('" + nom + "','" + note + "')"

    c.executescript(p)
```

```python
#Validation
    connexion.commit()


#Déconnexion
connexion.close()
```

Exercice 3:

- Soit un fichier intitulé concours.txt qui comporte les enregistrements relatifs aux candidats d'un concours. Chaque enregistrement est composé de : ncin, nom, prenom, age, decision. Les attributs sont séparés par (;).
- L'attribut decision prend un des valeurs suivantes : admis, refusé, ajourné
- Définir la fonction **saisir()** qui permet de remplir les données relatives aux candidats dans le fichier **concours.txt**.
- Définir la fonction **admis()** qui permet de créer le fichier admis.txt comportant les données relatives aux candidats admis.
- Afin de sélectionner en priorité les candidats admis et âgés plus que 30 ans, créer la fonction **attente ()** qui produira à partir du fichier **admis.txt** un nouveau fichier intitulé **attente.txt** comportant les données relatives aux candidats admis et âgés plus que 30 ans. Une ligne du fichier attente.txt comprend le ncin, le nom et le prenom d'un candidat séparés par (;)

Exercice 3:

```python
def saisir():
    new = "O"  # O -> oui ; N -> non
    fichier = open("concours.txt", "a")
    decision = {"a": "admis(e)", "r": "refuse(e)", "aj": "ajourne(e)"}
    while new == "O":
        cin = input("Saisir le Numero CIN : ")
        nom = input("Saisir le Nom : ")
        prenom = input("Saisir le prenom : ")
        age = input("saisir l age ")
        dec = input("saisir la decision a(admis(e))
                    r(refuse(e))
                    aj(ajourne(e)): ")
        ligne = cin+";"+nom+";"+prenom+";"+age+";"+decision[dec]+"\n"
        fichier.write(ligne)
        new = input("Saisir un nouveau candidat, (O / N) ?")
    fichier.close()
```

```python
def admis():
    fichier = open("concours.txt")
    dest = open("admis.txt", "a")
    for ligne in fichier:
        L = ligne.split(";")
        if L[4].strip() == "admis(e)":
            dest.write(ligne)
    fichier.close()
    dest.close()
```

```python
def attente():
    fichier = open("admis.txt")
    dest = open("attente.txt", "a")
    for ligne in fichier:
        L = ligne.split(";")
        if int(L[3]) >= 30:
            enreg = L[0]+";"+L[1]+";"+L[2]+"\n"
            dest.write(enreg)
    fichier.close()
    dest.close()
```

Exercice 3:

```
def statistiques(dec):
    fichier = open("concours.txt")
    L = fichier.readlines()
    L1 = []  # candidats admis
    L2 = []  # candidats refuses
    L3 = []  # candidats ajournes
    for ligne in L:
        L = ligne.split(";")
        if L[4].strip() == "admis(e)":
            L1.append(ligne)
        elif L[4].strip() == "refuse(e)":
            L2.append(ligne)
        else:
            L3.append(ligne)

    if dec == "admis":
            return (len(L1)/len(L))*100
        elif dec == "refuse":
            return (len(L2)/len(L))*100
        else:
            return (len(L3)/len(L))*100
```

```
def supprimer():
    fichier = open("admis.txt")
    candidat = []  # contient les candidats restants
    for ligne in fichier:
        L = ligne.split(";")
        if int(L[3]) < 30:
            candidat.append(ligne)
    fichier.close()
    # reecrire la nouvelle liste
    fichier = open("admis.txt", "w")
    fichier.writelines(candidat)
    fichier.close()
```

Exercice 4:

```
class Personne: #class Personne(): # class Personne(object):
    def __init__(self,nom, prenom, tel, email):
        self.nom = nom
        self.prenom = prenom
        self.tel = tel
        self.email = email

    def __str__(self):
        #doit retourner une chaine (pas print)
        ch ="{}, {} --Téléphone: {} -- Email :{}"
        return ch.format(self.nom,self.prenom, self.tel,self.email)
```

- Soit la classe Personne ci-dessous :
  - On souhaite créer une classe Travailleur qui hérite de Personne. Les attributs spécifiques au Travailleur sont nom de l'entreprise, adresse de l'entreprise et téléphone de l'entreprise où il est affilié.
  - Donner le code python relatif à la classe Travailler tout en tenant compte des informations déjà mentionnées. Modifier aussi la fonction __str__() afin qu'elle affiche les détails du Travailleur.
  - Développer la fonction calculSalaire de la classe Travailleur permettant de calculer le salaire d'une personne. Cette fonction prend comme paramètre le prix de l'heure travaillée et le nombre de jours travaillés par le travailleur et retourne le salaire. On note que le jour contient 8 heures de travail.
  - Ecrire le programme principal permettant de créer une personne p1 et deux travailleurs t1 et t2. La saisie du nom, de l'adresse et du téléphone de l'entreprise se fait par le programme principal.
  - Mettre à jour le programme principal afin d'afficher si les deux travailleurs ont le même salaire.
  - Construire la liste des travailleurs et l'afficher.

Exercice 4:

```python
class Personne: #class Personne(): # class Personne(object):
    def __init__(self,nom, prenom, tel, email):
        self.nom = nom
        self.prenom = prenom
        self.tel = tel
        self.email = email

    def __str__(self):
        #doit retourner une chaine (pas print)
        ch ="{}, {} --Téléphone: {} -- Email :{}"
        return ch.format(self.nom,self.prenom, self.tel,self.email)
```

```python
class Travailleur(Personne):
    def __init__(self, nom: str, prenom: str, date_naissance: str, entreprise_nom: str, en
        """
        Constructeur de la classe Travailleur, héritant de Personne.
        """
        super().__init__(nom, prenom, date_naissance)
        self.entreprise_nom = entreprise_nom
        self.entreprise_adresse = entreprise_adresse
        self.entreprise_tel = entreprise_tel

    def __str__(self):
        """
        Retourne une représentation textuelle de l'objet Travailleur.
        """
        return (
            super().__str__() +
            f", Entreprise: {self.entrep e_nom}, Adresse: {self.entreprise_adresse}, Tél
        )
```

```python
    def calculSalaire(self, prix_heure: float, nb_jours: int) -> float:
        """
        Calcule le salaire d'un travailleur.
        :param prix_heure: Le prix de l'heure de travail.
        :param nb_jours: Le nombre de jours travaillés.
        :return: Le salaire calculé.
        """
        heures_par_jour = 8
        salaire = prix_heure * nb_jours * heures_par_jour
        return salaire
```

Exercice 4:

```python
# Programme principal
if __name__ == "__main__":
    # Création d'une personne
    p1 = Personne("Dupont", "Jean", "15/03/1980")
    print("Personne créée :")
    print(p1)

    # Saisie des informations pour les travailleurs
    print("\nSaisie des informations pour deux travailleurs :")

    # Travailleur 1
    nom1 = input("Nom de l'entreprise pour travailleur 1 : ")
    adresse1 = input("Adresse de l'entreprise pour travailleur 1 : ")
    tel1 = input("Téléphone de l'entreprise pour travailleur 1 : ")
    t1 = Travailleur("Martin", "Paul", "22/05/1990", nom1, adresse1, tel1)
```

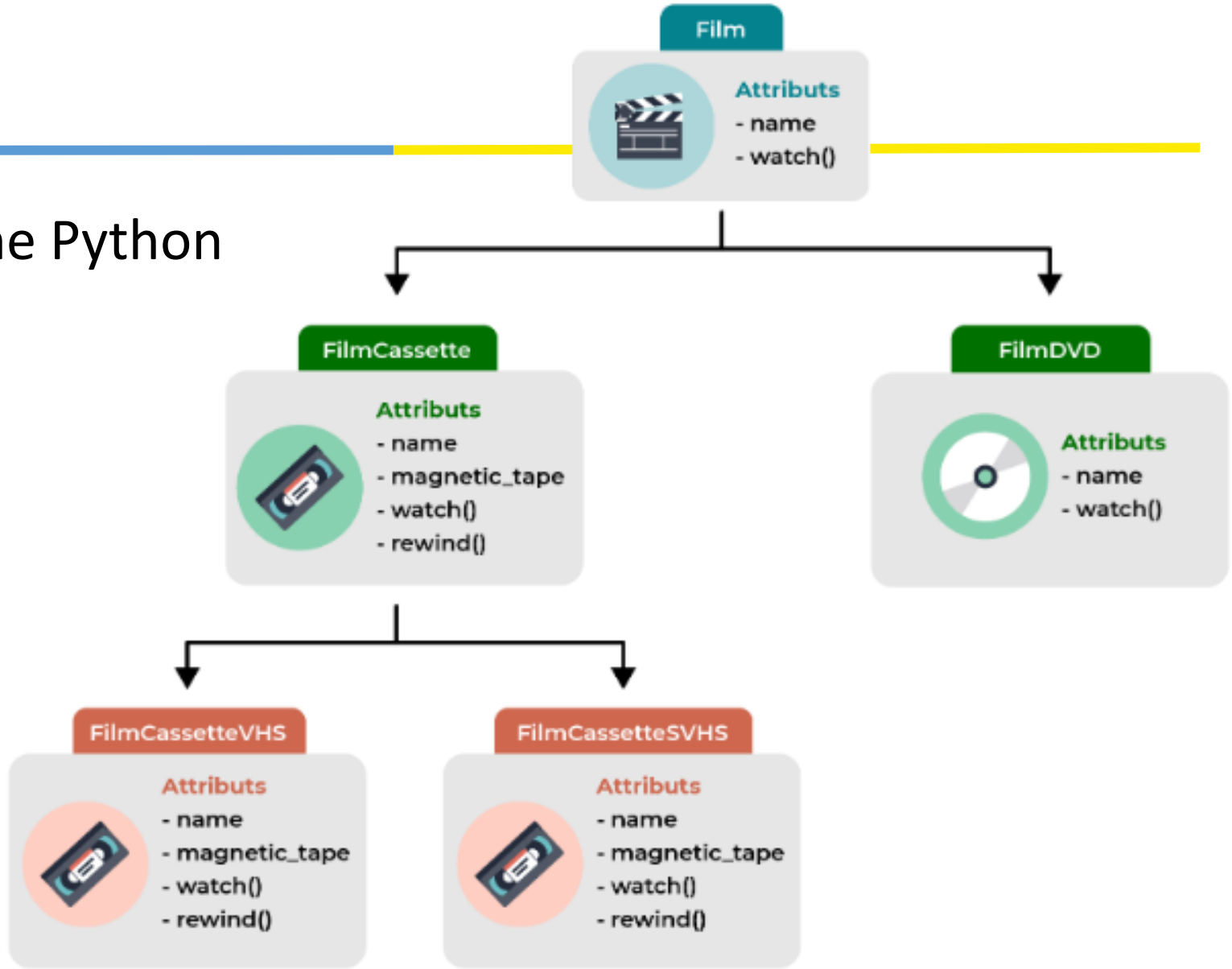
```python
nom2 = input("\nNom de l'entreprise pour travailleur 2 : ")
adresse2 = input("Adresse de l'entreprise pour travailleur 2 : ")
tel2 = input("Téléphone de l'entreprise pour travailleur 2 : ")
t2 = Travailleur("Durand", "Sophie", "10/12/1985", nom2, adresse2, tel2)

# Affichage des travailleurs
print("\nTravailleurs créés :")
print(t1)
print(t2)

# Calcul des salaires
print("\nCalcul des salaires :")
prix_heure1 = float(input("Prix de l'heure pour travailleur 1 : "))
nb_jours1 = int(input("Nombre de jours travaillés pour travailleur 1 : "))
salaire1 = t1.calculSalaire(prix_heure1, nb_jours1)
print(f"Salaire de travailleur 1 : {salaire1:.2f} €")
```

Exercice 4:

```python
prix_heure2 = float(input("\nPrix de l'heure pour travailleur 2 : "))
nb_jours2 = int(input("Nombre de jours travaillés pour travailleur 2 : "))
salaire2 = t2.calculSalaire(prix_heure2, nb_jours2)
print(f"Salaire de travailleur 2 : {salaire2:.2f} €")

# Vérification si les deux travailleurs ont le même salaire
if salaire1 == salaire2:
    print("\nLes deux travailleurs ont le même salaire.")
else:
    print("\nLes deux travailleurs n'ont pas le même salaire.")

# Création d'une liste des travailleurs
travailleurs = [t1, t2]

# Affichage de la liste des travailleurs
print("\nListe des travailleurs :")
for travailleur in travailleurs:
    print(travailleur)
```

Exercice 5:

- Donner un programme Python relatif à cette figure.



Exercice 6:

- Créez une base de données database.db et une table products avec les colonnes : id, name, category, price. 2. Insérez 6 produits appartenant à différentes categories de deux façons différentes : l'une en utilisant la commande executmany et l'autre en utilisant simplement la boucle for et la commande execute:

```python
products = [
    ("Laptop", "Electronics", 800),
    ("Smartphone", "Electronics", 600),
    ("Table", "Furniture", 120),
    ("Chair", "Furniture", 80),
    ("Notebook", "Stationery", 5),
    ("Pen", "Stationery", 1.5)
]
```

```python
# --- 1. Connexion à la base de données ---
conn = sqlite3.connect("database.db")
cursor = conn.cursor()

# --- 2. Création de la table products ---
cursor.execute("""
    CREATE TABLE IF NOT EXISTS products (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        category TEXT NOT NULL,
        price REAL NOT NULL
    )
""")

# Liste des produits (name, category, price)
products = [
    ("Laptop", "Electronics", 800),
    ("Smartphone", "Electronics", 600),
    ("Table", "Furniture", 120),
    ("Chair", "Furniture", 80),
    ("Notebook", "Stationery", 5),
    ("Pen", "Stationery", 1.5)
]
```

Insertion de plusieurs produits en même temps avec **executemany**

```python
cursor.executemany(
    "INSERT INTO products (name, category, price) VALUES (?, ?, ?)",
    products
)
```

Insertion produit par produit avec **execute**

```python
for name, category, price in products:
    cursor.execute(
        "INSERT INTO products (name, category, price) VALUES (?, ?, ?)",
        (name, category, price)
    )
```

```python
# Sauvegarde et fermeture
conn.commit()
conn.close()
```

Validation des opérations et fermeture de la base

```python
print("Base de données créée et produits insérés avec succès !")
```